

gpicaamera

March 20, 2013
14:23

Contents

1	Routine to set up camera views for use with NSTX Gas Puff Imaging Experiment	1
2	INDEX	6

1 Routine to set up camera views for use with NSTX Gas Puff Imaging Experiment

This file serves as an example *usr2ddetector.web* routine for the more complicated case of a 2-D imaging camera. The particular parameters used here are from the 2004 NSTX run campaign. More recent versions differ only in the numerical values of these parameters; the basic principles remain the same.

A key distinction between this and the approach used in simulating one dimensional imaging arrays (e.g., as in *btopdetector.web*) is that memory considerations prohibit computing the simulated image as part of the primary simulation (i.e., *flighttest*). For example, in this case, the *de_zone_frgs* array would have $64 \times 64 \times zn_num$ elements. For a typical 3-D GPI simulation, $zn_num \sim 10^5 \rightarrow 4 \times 10^8$ 64 bit real numbers or > 3 GB, quite a large amount of memory.

We instead compute a much lower resolution (represented by the *nx* and *ny* macro parameters below; both = 5) image during the run of *flighttest*; the resulting data are used only to check the simulation and camera geometry. The full camera image is computed subsequently via the *postdetector* routine. Because all of the constituent chord integrals are independent, each can be computed separately so that only *zn_num* elements are needed in *de_zone_frgs* at a time. This independence also allows this calculation can be parallelized for efficiency. The particular subroutine called from *postdetector* is *not detector_setup*, but *gpi_views*. An important implication of this is that *postdetector* is presently specific to these GPI applications.

Each of the camera pixels corresponds to a single chord specified by two points, as is described in the *Defining Radiation Detectors* section of the User's Manual. One of these points is the camera vertex, represented by the macros *x_p0*, *y_p0*, *z_p0* below. The “p” in the macro refers to the fact that the DEGAS 2 values for these coordinates are in a system rotated toroidally from the experimental coordinate system; this is explained further below.

The second point of each chord can be determined by whatever means are available. In all GPI simulations to date, these second points are obtained as a mapping between the camera's pixel coordinates and physical coordinates on a “target plane”. Conceptually, this is just a plane centered on the gas puff. In the NSTX case, it is defined in principle by a line (the gas manifold, a stainless steel tube) and a vector pointing at the center stack. In reality, it corresponds to a rectangular piece of metal bolted to the gas manifold.

Coordinates of arbitrarily located physical points on this “plane” are taken via a precision measuring arm, and a plane of the form

$$a_0x + a_1y + a_2z = 1$$

is fit to the resulting data. For convenience, the DEGAS 2 simulations place the center of the gas manifold at zero toroidal angle. Since the measurement arm data are referenced to a different location, a toroidal rotation *phi* must be applied to the target plane before use here. The end result is a second set of plane coefficients, $a'_0 \rightarrow a0_p$, etc. As noted above, this same rotation is applied to the coordinates of the camera vertex.

The gas puff for GPI experiments on Alcator C-Mod is provided by one or more capillaries at a single toroidal angle. In this case, the target plane is simply a vertical plane at that angle. The adaptation of this file to those experiments would require changing the values of the plane coefficients. Examples of such specifications can be found in *postdetector.web*, where the same target plane data are also used.

The mapping of points on this target plane to pixel coordinates is determined from an image of this same calibration target plane recorded by the camera (in NSTX, the process is rendered more complex by the optical system, although the final result is the same). The particular approach used in NSTX consists first

of using the resulting image to fit the mapping between the pixel coordinates (c_x, c_y) and spatial coordinates (R, Z) :

$$R = a_{Rx}c_x + a_{Ry}c_y + R_0$$

$$Z = a_{Zx}c_x + a_{Zy}c_y + Z_0,$$

where c_x ranges from cx_min to cx_max , and c_y ranges from cy_min to cy_max . Note that the coefficients in these equations do not appear below as variables or macros but simply as constants in the expressions for r_maj and z . Also note that relative spatial calibrations made during the campaign indicated that the camera view had shifted in the c_y direction, leading to the shift in that coordinate noted below.

But, in a 3-D simulation we need to know all three coordinates, (x, y, z) in the target plane for each pixel. Obviously, $z(c_x, c_y) = Z(c_x, c_y)$. The values of $x(c_x, c_y)$ and $y(c_x, c_y)$ are obtained from a quadratic equation derived by combining the definition of $R = \sqrt{x^2 + y^2}$ with the above equation for the target plane.

```
$Id:  gpica.camera.web,v 1.3 2009/11/25 19:41:30 dstotler Exp $
"gpica.camera.f" 1 ≡
  @m FILE 'gpica.camera.web'
```

The unnamed module.

```
"gpica.camera.f" 1.1 ≡
  ⟨ Functions and Subroutines 1.2 ⟩
```

GPI camera views.

```
"gpica
```

⟨ Functions and Subroutines 1.2 ⟩ ≡

subroutine *detector_setup*

implicit none *f77*

*de*_common

*zn*_common

implicit none *f90*

⟨ Memory allocation interface 0 ⟩

detector_total_views = *nx* * *ny*

var_alloc(*de_view_points*)

var_alloc(*de_view_algorithm*)

var_alloc(*de_view_halfwidth*)

var_alloc(*de_zone_frgs*)

de_grps = 0

de_view_size = 0

var_alloc(*de_view_tab*)

call *detector_setup_a*

return

end

See also sections 1.3 and 1.4.

This code is used in section 1.1.

Extension of the above subroutine. Statements actually making assignments to the detector pointer arrays (*de_zone_frgs* specifically) need to be separated from their allocation above so that their array indexing gets handled correctly.

⟨Functions and Subroutines 1.2⟩ +≡

subroutine *detector_setup_a*

define_varp(zone_frgs, FLOAT, de_zone_ind)

implicit_none_f77

de_common

zn_common

implicit_none_f90

integer *view, num, var, tab_index, spacing, ix, iy, i, zone, ix_max, iy_max*

integer *grp_views* *nx*ny*

real *var_min, var_max, mult*

declare_varp(zone_frgs)

⟨Memory allocation interface 0⟩

var_alloc(zone_frgs)

/ Use local variables for nx and ny so the arguments to gpi_views can have other values than those given by the macros at the top of this file. */*

ix_max = nx

iy_max = ny

do *ix = 1, nx*

do *iy = 1, ny*

*view = (iy - 1) * nx + ix*

call *gpi_views(ix, ix_max, iy, iy_max, vc_args(de_view_points_{view}, de_view_start),*
de_view_halfwidth_{view}, de_view_algorithm_{view}, zone_frgs)

/ The order of the indices in de_zone_frgs has the views listed consecutively, not the zones.*

*Hence, cannot use it directly as the argument for this call. */*

do *zone = 1, zn_num*

de_zone_frgs_{zone,view} = zone_frgs_{zone}

end do

end do

end do

var_free(zone_frgs)

*num = nx * ny*

var = de_var_unknown

tab_index = zero

var_min = zero

var_max = zero

mult = zero

spacing = de_spacing_unknown

do *i = 1, num*

grp_views_i = i

end do

call *de_grp_init('GPI_chords', num, var, tab_index, var_min, var_max, mult, spacing, grp_views)*

return

end

Details of the GPI camera views. These are separated here so that they can be called by a post-processing routine, separate from the detector class arrays.

⟨ Functions and Subroutines 1.2 ⟩ +=

```

subroutine gpi_views(ix, ix_max, iy, iy_max, vc_dummy(points), halfwidth, algorithm, zone_frgs)
  implicit none f77
  zn_common
  implicit none f90

  integer ix, ix_max, iy, iy_max    // Input

  integer algorithm    // Output
  real halfwidth

  vc_decl(pointsde_view_start:de_view_end)
  real zone_frgszn_num

  real delta_cx, delta_cy, a0_p, a1_p, a2_p, cx, cy,    /* Local */
        r_maj, z, disc, x_p, y_p, z_p

  assert(ix_max > 1)
  assert(iy_max > 1)
  delta_cx = (cx_max - cx_min) / areal(ix_max - 1)
  delta_cy = (cy_max - cy_min) / areal(iy_max - 1)
  /* Rotate Ricky's "target plane" to my coordinate system. The "p" denotes "prime". */
  a0_p = a0 * cos(phi) + a1 * sin(phi)
  a1_p = a1 * cos(phi) - a0 * sin(phi)
  a2_p = a2
  cx = cx_min + areal(ix - 1) * delta_cx
  cy = cy_min + areal(iy - 1) * delta_cy
  /* This was the original mapping provided by Ricky Maqueda. */
  @if 0
    r_maj = const(1., -3) * (-const(0.7555) * cx - const(3.5159) * (cy + const(2.67)) + const(1.604, 3))
    z = const(1., -3) * (const(2.8476) * cx - const(0.5254) * (cy + const(2.67)) + const(1.31, 2))
  @endif
  /* Subsequently concluded that a 6 pixel was appropriate for these shots: */
  r_maj = const(1., -3) * (-const(0.7555) * cx - const(3.5159) * (cy + const(6.0)) + const(1.604, 3))
  z = const(1., -3) * (const(2.8476) * cx - const(0.5254) * (cy + const(6.0)) + const(1.31, 2))
  /* Solve for corresponding Cartesian coordinates in my system using the rotated target plane and
      $R^2 = x^2 + y^2$ . */
  z_p = z
  disc = (a0_p2 + a1_p2) * r_maj2 - (one - a2_p * z_p)2
  assert(disc > zero)
  x_p = (a0_p * (one - a2_p * z_p) + a1_p * sqrt(disc)) / (a0_p2 + a1_p2)
  y_p = (a1_p * (one - a2_p * z_p) - a0_p * sqrt(disc)) / (a0_p2 + a1_p2)    /* The "4" here accounts
     for the fact that there are two computed variables, x_p and y_p, and that they are squared. */
  assert(abs(r_maj2 - x_p2 - y_p2) < const(4.0) * epsilon)
  algorithm = de_algorithm_circular
  halfwidth = const(3.9, -1) * PI / const(1.8, 2)
  vc_set(pointsde_view_start, x_p0, y_p0, z_p0)
  vc_set(pointsde_view_end, x_p, y_p, z_p)
  call detector_view_setup(vc_args(pointsde_view_start), halfwidth, algorithm, zone_frgs)

  return
end

```

2 INDEX

abs: 1.4.
algorithm: 1.4.
areal: 1.4.
assert: 1.4.
a0: 1.2, 1.4.
a0_p: 1, 1.4.
a1: 1.2, 1.4.
a1_p: 1.4.
a2: 1.2, 1.4.
a2_p: 1.4.
btopdetector: 1.
const: 1.2, 1.4.
cos: 1.4.
cx: 1.4.
cx_max: 1, 1.2, 1.4.
cx_min: 1, 1.2, 1.4.
cy: 1.4.
cy_max: 1, 1.2, 1.4.
cy_min: 1, 1.2, 1.4.
de_algorithm_circular: 1.4.
de_common: 1.2, 1.3.
de_grp_init: 1.3.
de_grps: 1.2.
de_spacing_unknown: 1.3.
de_var_unknown: 1.3.
de_view_algorithm: 1.2, 1.3.
de_view_end: 1.4.
de_view_halfwidth: 1.2, 1.3.
de_view_points: 1.2, 1.3.
de_view_size: 1.2.
de_view_start: 1.3, 1.4.
de_view_tab: 1.2.
de_zone_frgs: 1, 1.2, 1.3.
de_zone_ind: 1.3.
declare_varp: 1.3.
define_varp: 1.3.
delta_cx: 1.4.
delta_cy: 1.4.
detector_setup: 1, 1.2.
detector_setup_a: 1.2, 1.3.
detector_total_views: 1.2.
detector_view_setup: 1.4.
disc: 1.4.
epsilon: 1.4.
FILE: 1.
flighttest: 1.
FLOAT: 1.3.
gpi_views: 1, 1.3, 1.4.
grp_views: 1.3.
halfwidth: 1.4.
i: 1.3.
implicit_none_f77: 1.2, 1.3, 1.4.
implicit_none_f90: 1.2, 1.3, 1.4.
ix: 1.3, 1.4.
ix_max: 1.3, 1.4.
iy: 1.3, 1.4.
iy_max: 1.3, 1.4.
mult: 1.3.
num: 1.3.
nx: 1, 1.2, 1.3.
ny: 1, 1.2, 1.3.
one: 1.4.
phi: 1, 1.2, 1.4.
PI: 1.2, 1.4.
points: 1.4.
postdetector: 1.
r_maj: 1, 1.4.
sin: 1.4.
spacing: 1.3.
sqrt: 1.4.
tab_index: 1.3.
usr2ddetector: 1.
var: 1.3.
var_alloc: 1.2, 1.3.
var_free: 1.3.
var_max: 1.3.
var_min: 1.3.
vc_args: 1.3, 1.4.
vc_decl: 1.4.
vc_dummy: 1.4.
vc_set: 1.4.
view: 1.3.
web: 1.
x_p: 1.4.
x_p0: 1, 1.2, 1.4.
y_p: 1.4.
y_p0: 1, 1.2, 1.4.
z: 1.4.
z_p: 1.4.
z_p0: 1, 1.2, 1.4.
zero: 1.2, 1.3, 1.4.
zn_common: 1.2, 1.3, 1.4.

zn_num: 1, 1.3, 1.4.

zone: 1.3.

zone_frgs: 1.3, 1.4.

⟨ Functions and Subroutines 1.2, 1.3, 1.4 ⟩ Used in section 1.1.

⟨ Memory allocation interface 0 ⟩ Used in sections 1.3 and 1.2.

COMMAND LINE: "fweave -f -i! -W[-ykw800 -ytw40000 -j -n/
/u/dstotler/degas2/src/gpicaamera.web".

WEB FILE: "/u/dstotler/degas2/src/gpicaamera.web".

CHANGE FILE: (none).

GLOBAL LANGUAGE: FORTRAN.